Tender Requirement Matching: Intern Assignment

Time box: 2 weeks | Submission: repo + short write-up + 15-min walkthrough

Background

A GeM tender can list hundreds of technical requirements. Before bidding, someone has to check whether a given manufacturer's products can actually meet each one, by digging through catalogues and datasheets. It is slow and error-prone. Your task is to build a system that does this automatically and, most importantly, can justify every decision with evidence.

The task

Take a GeM tender and a set of manufacturers in the same product category (for example, for a valves and pumps tender: KSB, Duncan, Delval Flow, Intervalve Poonawala, or others you find relevant). Evaluate every technical requirement in the tender against what each manufacturer can supply, and produce a compatibility report.

How you design it is up to you. We care about the quality of the output and the reasoning behind your choices, not about which tools you use.

Inputs

- **Tenders:** the 10–15 GeM tenders you already have access to. Use them to develop and test. We will evaluate on 2 tenders of our choice.
- **Manufacturer knowledge:** nothing is provided. You decide which manufacturers are relevant for each tender category and collect their publicly available product catalogues, datasheets, brochures and spec sheets yourself. Deciding what data is needed, where to find it, and how to organise it is part of the assignment.

Required output

For each requirement in the tender, produce:

Requirement	Manufacturer capability	Status	Score	Evidence
SS316 body	KSB: SS316 available	Full match	100%	Datasheet, p.2, "Body: SS316"
1500 RPM	Max 1200 RPM	Not met	0%	Datasheet, p.3
IP68	IP67 available	Partial	Partial	Brochure, p.1

Statuses to support: fully matched, partially matched, missing capability, and unsupported requirement. We deliberately do not define the last two precisely. Decide what each should mean, and state your definitions in your write-up.

Summary for the whole tender:

- Overall compatibility % for each manufacturer, with a clear explanation of how it is calculated, and a ranking of which manufacturer fits the tender best.
- Counts and lists for each status.
- Every claim must point to a source: document, page and the supporting text.

Deliver the results in a machine-readable format and in a form a bid manager could read. We should be able to run your system on a new tender and get a report without touching your code.

Things to think about

Real tenders are messy. A good solution will have a considered answer to questions like:

- Requirements with units, ranges, or direction ("min", "max", "≥").

- A single line that contains several requirements.
- Requirements that are commercial, legal or too vague to verify technically.
- Things the collected catalogues say nothing about. Is that a failure or a guess?
- Catalogues that are scanned, table-heavy, or cover a whole product range rather than one model.
- Conflicting or outdated information across documents.
- How sure is your system, and when should a human review it?

Deliverables

- The collected manufacturer data, with a note on where each document came from.
- **Backend:** Python with FastAPI, written as modular, well-structured code (clear separation of API routes, business logic, data access and configuration, not one big file).
- **Frontend:** React preferred; Streamlit is acceptable. It should let a user upload a tender, run the matching, and view the report with evidence.
- **Docker:** two separate images, one for the backend and one for the frontend, both pushed to Docker Hub as public images.
- **Code repository:** all code pushed to a private GitHub repository, with us added as collaborators so we can review it.
- **README:** setup instructions, plus exact commands to pull and run both images (including how they connect, required environment variables and ports) so we can get the whole system running without reading your code. Include example output on one tender.
- Short write-up (max 2 pages): your approach, assumptions, definitions, known failure cases, and what you would do with another week.
- Evidence of how you tested it, including how you measured accuracy on the tenders you have.

Stretch (optional): confidence scores or a human-review flag, storing past runs, handling a new product category with no code changes, an automated evaluation harness.

How we will evaluate

We will pick 2 tenders, run your system, and check your output requirement by requirement against the catalogue pages you cite.

Area	Weight	What we look for
Manufacturer data coverage	20%	Right manufacturers, right documents, well organised and traceable
Matching accuracy	25%	Correct status per requirement
Evidence quality	20%	Citations exist, are accurate, and nothing is made up
Handling of hard cases	10%	Units, partial matches, compound and unverifiable requirements
Engineering quality	10%	Runs cleanly, readable code, sensible error handling
Write-up and self-evaluation	15%	Clear reasoning, honest limitations, own testing

A system that says “not enough evidence” when it is unsure will score better than one that guesses confidently. You may use any tools or AI assistance, but you must be able to explain and defend every part of your solution in the walkthrough.